CS3400 - Tutorial One

1. Distinguish between an **Abstract Data Type (ADT)** and a **data structure**.
2. A Stack ADT can be implemented using either an array or a linked list. Explain why changing the implementation does not change the ADT.
3. Give **one reason** why separating an ADT from its implementation is useful when designing software.
4. Consider the following requirement:

   > A university needs to store student numbers and corresponding student names. The system must frequently retrieve a student's name using their student number.

   a. Identify a suitable ADT for this problem.
   b. Suggest a suitable C++ STL container.
   c. Suggest an appropriate underlying approach/algorithm for performing the lookup.
   d. Explain why choosing an appropriate data structure/container can affect the efficiency of the solution.
5. Determine the asymptotic time complexity of each code fragment. State the basic operation and briefly justify your answer.
   a.
   ```
   for (int i = 0; i < n; i++) {
    cout << i;
   }
   ```
   b.
   ```
   for (int i = 0; i < n; i++) {
   cout << i;
   }
   for (int j = 0; j < n; j++) {
   cout << j;
   }
   ```
   c.
   ```
   for (int i = 0; i < n; i++) {
     for (int j = 0; j < n; j++) {
   cout << i + j;
    }
   }
   ```

6. Consider the following

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 10; j++) {
        cout << i + j;
    }
}
```

   a. What is its basic operation?
   b. What is its input size?
   c. How many times does the basic operation execute in terms of n?
   d. Determine the time complexity.
   e. A student argues: “There are two nested for loops, so the algorithm must be $O(n^2)$. Is the student correct? explain.

7. Consider the following

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) {
        cout << "*";
    }
}
```

   a. Express the total number of executions of the basic operation as a summation.
   b. Use the appropriate summation formula to simplify your expression.
   c. Determine the asymptotic complexity.

8. Consider the following

```
int i = n;
while (i > 1) {
    i = i / 2;
}
```

   a. Explain what happens to *i* after each iteration.
   b. Approximately how many iterations occur as a function of *n*?

c. Determine the time complexity.
d. Explain why this is fundamentally different from an algorithm that reduces *i* by 1 on every iteration.

9. Consider an algorithm whose running time is:

$$T(n) = 3n^2 + n + 8$$

a. Give its Big-O, upper bound.
b. Give its Big-Ω, lower bound.
c. Give its Big-Θ, tight bound.

10. Answer as true or false and justify your answer:
a. O(5n)=O(n)
b. $O(n^2+n)=O(n)$
c. An algorithm with O(n) running time is necessarily faster in practice than every algorithm with $O(n^2)$ running time for every possible input size.
d. An algorithm can have O(n) time complexity and O(1) auxiliary space complexity.

11. Suppose you have 1,000,000 student numbers.

You need to determine whether a particular student number exists.

You are considering:

- Algorithm A: Linear search through an unsorted array.
- Algorithm B: Binary search through a sorted array.
- Algorithm C: Hash-table lookup.

a. Give the typical time complexity of each approach.
b. Which approach would generally provide the fastest lookup for repeated searches?
c. What trade-off may be involved in choosing the hash-table solution?

12. Consider the following

```
vector<int> copy;

for (int i = 0; i < n; i++) {

   copy.push_back(numbers[i]);

}
```

a. Determine the time complexity of the loop.
b. Determine the additional/auxiliary space complexity.
c. Explain why this example illustrates that time and space complexity are different measures.